ID NO:2400030040

Name: N. Sri Aravind.

#SKILL - 2 -> Hibernate CRUD Operations

- **Basic knowledge of Java**
- **Understanding of classes, objects, and OOP**
- **Familiarity with SQL basics**

A retail inventory system needs to store product details such as productId, name, description, price, and quantity. The admin should be able to add new products, retrieve product information, update price or quantity, and delete discontinued products. Your task is to implement complete CRUD operations using Hibernate with proper entity mapping using JPA annotations

Tasks:

1. Create a Product entity with fields: id (primary key), name, description, price, quantity.

2. Configure Hibernate and map the Product entity to a table using JPA annotations.

3. Insert multiple Product records into the database.

4. Retrieve a product using its id.

5. Update the price or quantity of any selected product.

6. Delete a product record by id if it is discontinued.

7. Note: a. Implement Hibernate ID generation strategies (AUTO, IDENTITY, SEQUENCE) inside the entity to observe how primary key values differ.

b. The ID can be manual or auto generated, based on your chosen strategy.

8. Push the Hibernate project into a single GitHub repository.

1) Product.java

```
package com.example;

import javax.persistence.*;

@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private int id;
    private String name;
    private String description;
    private double price;
    private int quantity;

    public Product() {
    }

    public Product(String name, String description, double price, int quantity) {
        this.name = name;
        this.description = description;
        this.price = price;
        this.quantity = quantity;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getDescription() {
        return description;
    }

    public double getPrice() {
        return price;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setPrice(double price) {
        this.price = price;
    }

     public void setQuantity(int quantity) {
```

```
            this.quantity = quantity;
        }
    }
```

2)MainApp.java

```
package com.example;


import java.util.Scanner;

import org.hibernate.*;

import org.hibernate.cfg.Configuration;

public class MainApp {

    static SessionFactory factory = new Configuration().configure().buildSessionFactory();

    static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {

        int choice;


        do {

            System.out.println("\n===== PRODUCT MANAGEMENT =====");

            System.out.println("1. Insert Product");

            System.out.println("2. View Product");

            System.out.println("3. Update Product");

            System.out.println("4. Delete Product");

            System.out.println("5. Exit");

            System.out.print("Enter choice: ");

            choice = sc.nextInt();


            switch (choice) {

                case 1:

                    insertProduct();

                    break;

                case 2:

                    viewProduct();

                    break;

                case 3:
```

```
            updateProduct();
            break;
        case 4:
            deleteProduct();
            break;
        case 5:
            factory.close();
            System.out.println("Exiting Application...");
            break;
        default:
            System.out.println("Invalid Choice!");
    }
  } while (choice != 5);
}



static void insertProduct() {
  Session session = factory.openSession();
  Transaction tx = session.beginTransaction();

  System.out.print("Enter Name: ");
  sc.nextLine();
  String name = sc.nextLine();

  System.out.print("Enter Description: ");
  String desc = sc.nextLine();

  System.out.print("Enter Price: ");
  double price = sc.nextDouble();

  System.out.print("Enter Quantity: ");
  int qty = sc.nextInt();
```

```
    Product p = new Product(name, desc, price, qty);
    session.save(p);

    tx.commit();
    session.close();
    System.out.println("Product Inserted Successfully!");
}


static void viewProduct() {
    Session session = factory.openSession();

    System.out.print("Enter Product ID: ");
    int id = sc.nextInt();

    Product p = session.get(Product.class, id);

    if (p != null) {
        System.out.println("ID: " + p.getId());
        System.out.println("Name: " + p.getName());
        System.out.println("Price: " + p.getPrice());
        System.out.println("Quantity: " + p.getQuantity());
    } else {
        System.out.println("Product Not Found!");
    }
    session.close();
}


static void updateProduct() {
    Session session = factory.openSession();
    Transaction tx = session.beginTransaction();
```

```
    System.out.print("Enter Product ID to Update: ");
    int id = sc.nextInt();

    Product p = session.get(Product.class, id);

    if (p != null) {
        System.out.print("Enter New Price: ");
        p.setPrice(sc.nextDouble());

        System.out.print("Enter New Quantity: ");
        p.setQuantity(sc.nextInt());

        session.update(p);
        tx.commit();
        System.out.println("Product Updated Successfully!");
    } else {
        System.out.println("Product Not Found!");
    }

    session.close();
}


static void deleteProduct() {
    Session session = factory.openSession();
    Transaction tx = session.beginTransaction();

    System.out.print("Enter Product ID to Delete: ");
    int id = sc.nextInt();

    Product p = session.get(Product.class, id);

    if (p != null) {
```

```
            session.delete(p);
            tx.commit();
            System.out.println("Product Deleted Successfully!");
        } else {
            System.out.println("Product Not Found!");
        }

        session.close();
    }
}
```

3)hibernate.cfg.xml

```
<hibernate-configuration>
    <session-factory>
        <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/fsaddb</property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password">Aravind@17</property>
        <property name="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</property>
        <property name="hibernate.hbm2ddl.auto">update</property>
        <property name="hibernate.show_sql">true</property>
        <mapping class="com.example.Product"/>
    </session-factory>
</hibernate-configuration>
```

Output:

```
===== PRODUCT MANAGEMENT =====
1. Insert Product
2. View Product
3. Update Product
4. Delete Product
5. Exit
Enter choice: 1
Enter Name: Laptop
Enter Description: Dell i5 12th Gen
Enter Price: 55000
Enter Quantity:  10
Hibernate: select next_val as id_val from hibernate_sequence for update
Hibernate: update hibernate_sequence set next_val= ? where next_val=?
Hibernate: insert into products (description, name, price, quantity, id) values (?, ?, ?, ?, ?)
Product Inserted Successfully!

===== PRODUCT MANAGEMENT =====
1. Insert Product
2. View Product
3. Update Product
4. Delete Product
5. Exit
Enter choice: 2
Enter Product ID: 1
Hibernate: select product0_.id as id1_0_0_, product0_.description as descript2_0_0_, product0_.name as name3_0_0_, produ
ID: 1
Name: Laptop
Price: 75000.0
Quantity: 10
```

```
==== PRODUCT MANAGEMENT =====
. Insert Product
. View Product
. Update Product
. Delete Product
. Exit
nter choice: 3
nter Product ID to Update: 1
ibernate: select product0_.id as id1_0_0_, product0_.description as descript2_0_0_, product0_.name as na
nter New Price: 52000
nter New Quantity: 8
ibernate: update products set description=?, name=?, price=?, quantity=? where id=?
roduct Updated Successfully!

==== PRODUCT MANAGEMENT =====
. Insert Product
. View Product
. Update Product
. Delete Product
. Exit
nter choice: 2
nter Product ID: 1
ibernate: select product0_.id as id1_0_0_, product0_.description as descript2_0_0_, product0_.name as na
D: 1
ame: Laptop
rice: 52000.0
uantity: 8
```

```
===== PRODUCT MANAGEMENT =====
1. Insert Product
2. View Product
3. Update Product
4. Delete Product
5. Exit
Enter choice: 4
Enter Product ID to Delete: 1
Hibernate: select product0_.id as id1_0_0_, product0_.description as descript2_0_0_, product0_.name as name3_0_0_,
Hibernate: delete from products where id=?
Product Deleted Successfully!

===== PRODUCT MANAGEMENT =====
1. Insert Product
2. View Product
3. Update Product
4. Delete Product
5. Exit
Enter choice: 5
Feb 06, 2026 10:42:52 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PoolSta
INFO: HHH10001008: Cleaning up connection pool [jdbc:mysql://localhost:3306/fsaddb]
Exiting Application...
```

Git Repository:

https://github.com/Aravind2k6/2400030040-Fsad-WorkBook.git

VIVA QUESTIONS:

1. What is ORM and why is it used in application development?

A. ORM (Object Relational Mapping) is a technique that maps database tables to objects in programming languages.
It is used to simplify database operations by allowing developers to interact with the database using objects instead of writing SQL queries.

2.Which annotation is used to mark a class as a JPA entity?

A. The annotation used to mark a class as a JPA entity is @Entity.
It tells the JPA provider that the class should be mapped to a database table.

3. Why must every Hibernate entity have a primary key?

A. Every Hibernate entity must have a primary key to uniquely identify each record in the database.
It helps Hibernate track, update, and manage objects properly..

4. How do you update an existing record using Hibernate?

A. To update an existing record in Hibernate, retrieve the object using its primary key, modify its fields, and call update() or commit the transaction.
Hibernate automatically synchronizes the changes with the database during transaction commit.

5. What is the difference between get() and load() in Hibernate?

A. get() immediately fetches the object from the database and returns null if the record is not found.
load() returns a proxy object and throws an exception if the record does not exist.